



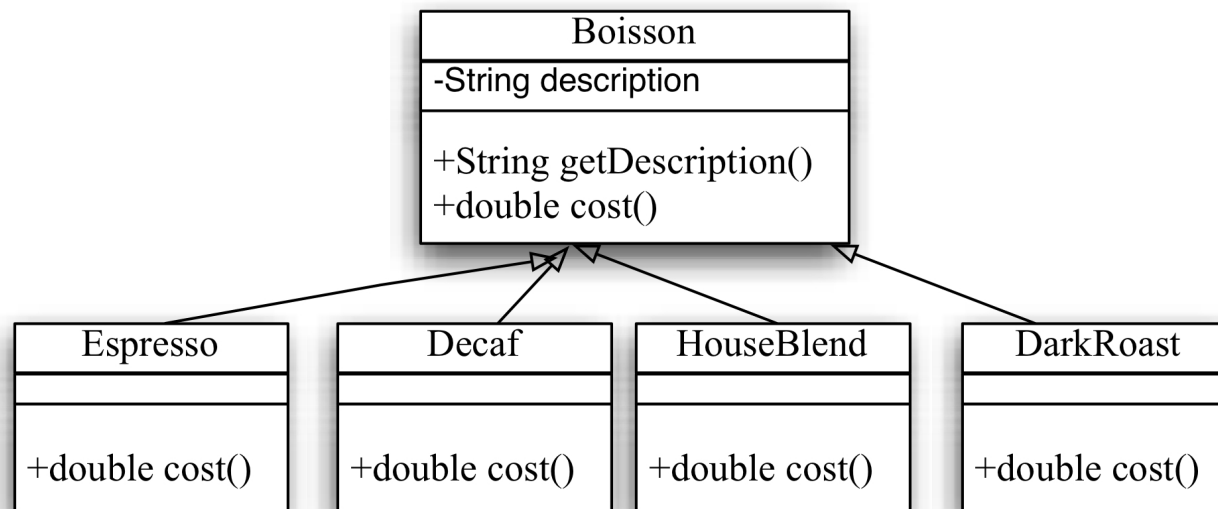
Master 1: Réseaux et Sécurité Informatique
Programmation Orienté Objet Avancée

Decorator

Un Design Pattern Structurel

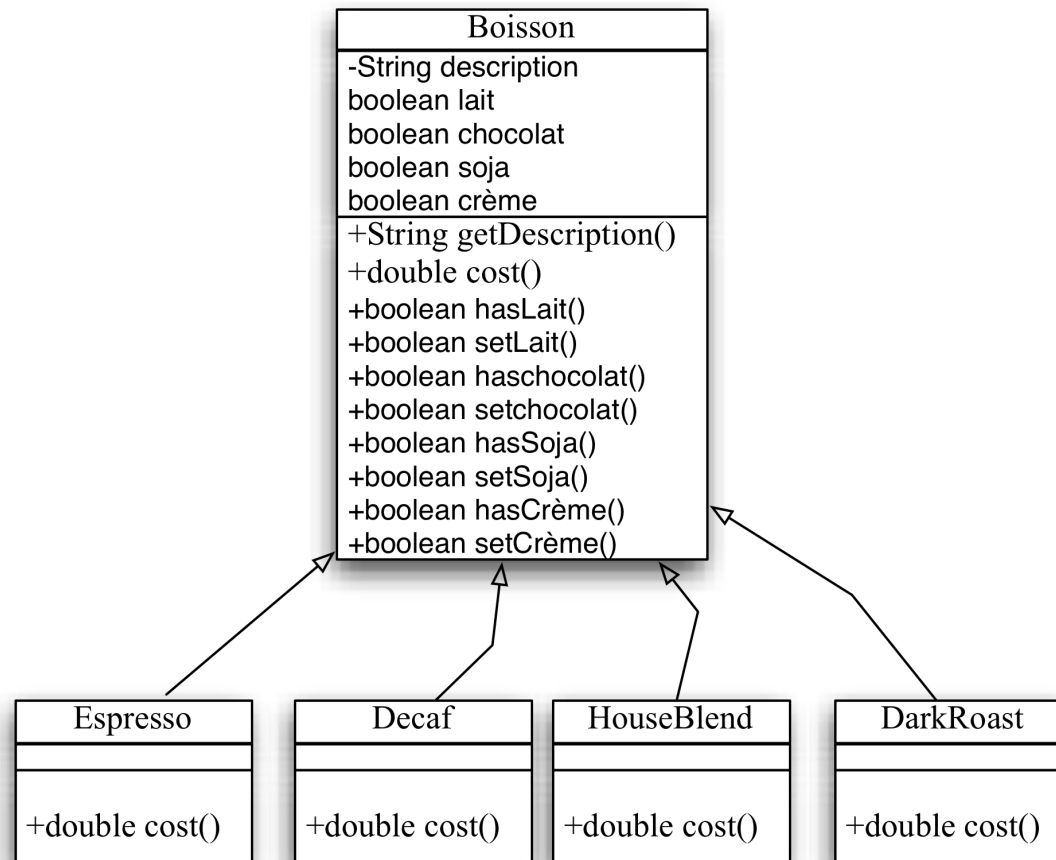
Exemple introductif

- Soit le diagramme suivant



- On voudrait rajouter des condiments à ces différents cafés ; par exemple lait, chocolat, crème, soja,.....avec toutes les combinaisons possibles, puis calculer le coût total de chaque boisson....
- Si l'on crée une classe pour chaque combinaison possible, on aura une explosion de classe, en plus:
 - Si le prix du lait augmente, il sera ardu de le modifier partout où il apparaît
 - Si l'on rajoute un nouveau condiment la mise à jour sera difficile

Solution évitant l'explosion de classes



Exercice: écrire les méthodes `cost()` de la classe **Boisson** et celle de ses différentes sous-classes

Problèmes liés à la solution de 5 classes

- Si le prix des condiments change, on doit modifier le code (variables d'instances et méthode `cost()`)
- Si on doit rajouter de nouveaux condiments (eg Caramel), on doit rajouter de nouvelles méthodes (`has` et `set Caramel`) et modifier le code de la méthode `cost()`
- Si l'on rajoute une nouvelle boisson (par eg. Thé, Thé glacé) les condiments ne seront pas appropriés, et malgré cela la classe Thé doit hériter toutes les méthodes des condiments

Résolution des problèmes précédant: le principe « Open Close »

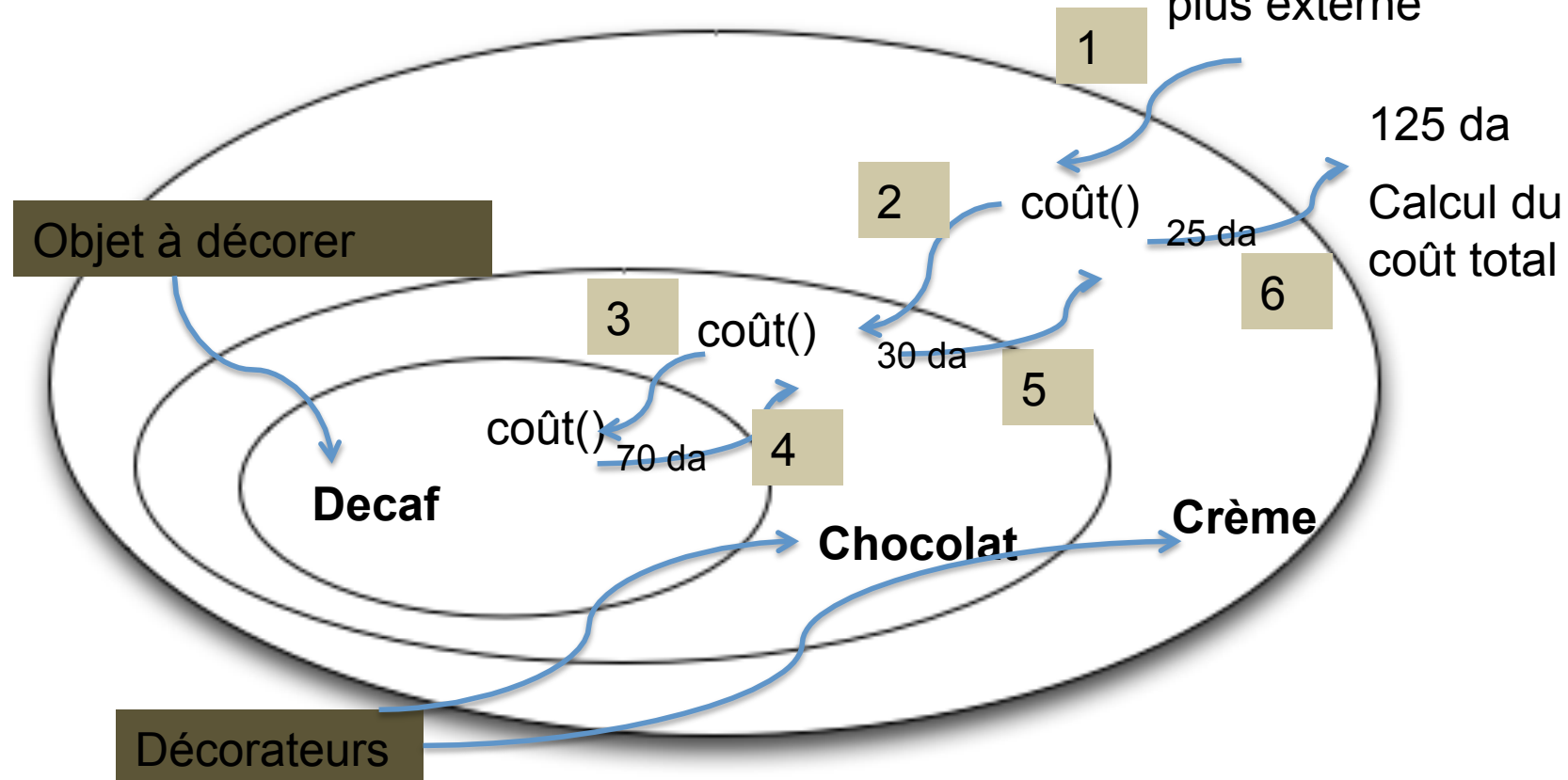
- Ce principe de la programmation orientée objet stipule: les classes doivent être ouvertes à l'extension mais fermés à la modification.
- Remarque: il ne faut pas appliquer ce principe partout, sinon il y aura complexité dans la gestion des classes.
- L'expérience et la maîtrise du domaine de travail feront le meilleur guide.

Le Pattern Decorator

- Le pattern Decorator est un parfait exemple d'application pour le principe Open Close
- On va l'appliquer à l'exemple Cafétéria
- Si un client demande un decaf+chocolat+crème:
 - On va prendre un objet decaf
 - On le décore avec un objet chocolat
 - Ensuite on le décore avec un objet crème
 - Appeler la méthode cost() et grâce à la délégation, calculer le coût des différents condiments

Représentation graphique

On commence par appeler le coût sur le décorateur le plus externe



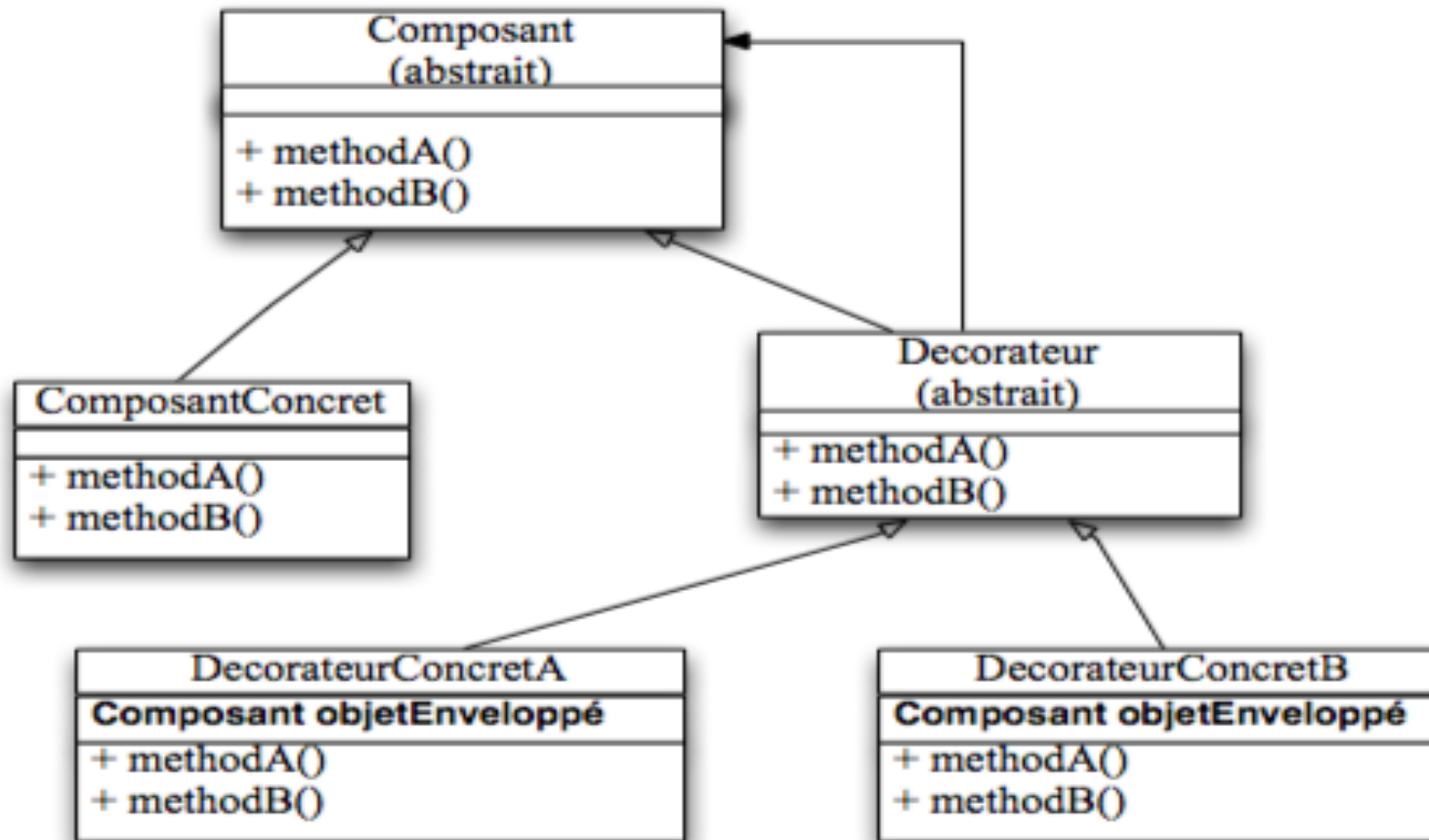
Résultats d'application du décorateur

- Le décorateur doit refléter le type de l'objet qu'il décore → décorateur et objet ont le même super type
- On peut utiliser un ou plusieurs décorateurs pour envelopper un objet
- Le décorateur rajoute son comportement avant(et/ou) après avoir délégué à l'objet qu'il décore pour faire le reste du travail

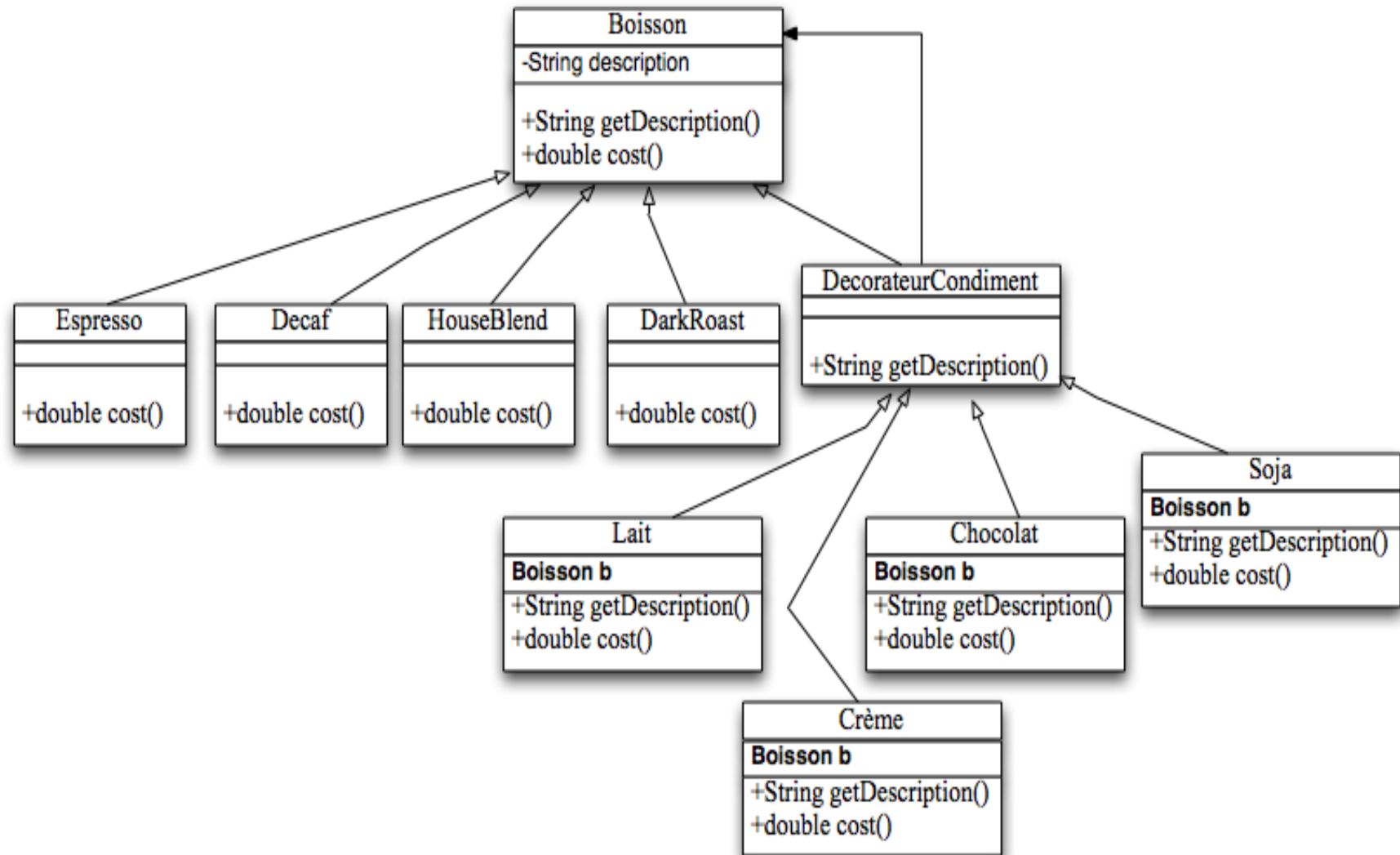
Définition du Pattern Decorator

- Decorator attache dynamiquement de nouvelles responsabilités à un objet
- Il fournit une alternative(flexible) à l'héritage pour étendre les fonctionnalités

Decorator: Diagramme de classe



Decorator: application à l'exemple cafétéria



Codage de l'exemple cafétéria

Les deux classes abstraites de base

```
public abstract class Boisson{  
    String description = "Boisson inconnue";  
public String getDescription() { return  
    description; }  
public abstract double cost(); }  
  
public abstract class DecorateurCondiment  
    extends Boisson{  
    public abstract String getDescription();  
}
```

Codage de l'exemple cafétéria(suite)

Les boissons à décorer

```
public class Espresso extends Boisson{  
public Espresso() {  
description = "Espresso Bello!";  
}  
public double cost() { return 100.50; } }
```

- Le reste des boissons auront le même style de codage

Codage de l'exemple cafétéria(suite)

Les condiments décorateurs

```
public class Chocolat extends DecorateurCondiment {  
    Boisson boisson;  
    public Chocolat(Boisson boisson) {  
        this.boisson = boisson;  
    }  
    public String getDescription() {  
        return boisson.getDescription() + “, Chocolat”;  
    }  
    public double cost() {  
        return 30 + boisson.cost();  
    }  
}
```

Codage de l'exemple cafétéria(suite) création des objets

```
public class StarCoffee {  
public static void main(String args[]) {  
  
    //un client demandant un espresso sans condiments  
    Boisson boisson= new Espresso(); System.out.println(boisson.getDescription()  
    + " DA" + boisson.cost());  
  
    //un client demandant un DarkRoast avec double chocolat et crème  
    Boisson boisson2 = new DarkRoast();  
    boisson2 = new Chocolat (boisson2); boisson2 = new Chocolat(boisson2); boisson2 = new Crème(boisson2);  
  
    //on pourra aussi écrire Boisson boisson2=new Crème(new Chocolat(new Chocolat(new DarkRoast())))  
  
    System.out.println(boisson2.getDescription() + "DA" + boisson2.cost());  
  
    //un client demandant un HouseBlend avec soja, chocolat et crème  
  
    Boisson boisson3 = new HouseBlend();  
    boisson3 = new Soja(boisson3); boisson3 = new Chocolat(boisson3); boisson3 = new Crème(boisson3);  
  
    System.out.println(boisson3.getDescription() + " DA" + boisson3.cost());  
}}
```